CONTROLLER ROUTER

Deployment Guide

Package the library and optional UNO Q runtime,
with safe upgrades and recovery.

VIRTUALGLOVE / R.O.B. VISION / MAKER CONTROLLERS

Current source edition | 27 October 2026

This guide covers packaging and deploying Controller Router inside your own controller project. The Integration Guide describes the design; the Technical Reference defines the APIs.

Install the console library

The repository's `pyproject.toml` packages `router_shared` as `maker-controller-router`. It declares Python 3.10 or newer. From a local checkout, install it in your project's virtual environment:

```
python3 -m venv .venv
. .venv/bin/activate
python3 -m pip install .
```

The library has no third-party Python runtime dependencies. Live routing requires Linux input devices, uinput permission, EmulationStation mappings for ordinary pads, and RetroArch. A macOS development machine can inspect and test the API but cannot create Linux uinput outputs.

The wheel is Python-only and does not need a CPU-specific build. It does not install service units, a receiver, game launch hooks, a web setup interface, or UNO Q firmware. Package those integration assets in your project.

Package a complete integration

1. Include the shared library through your dependency package or a maintained vendored copy.
2. Install your receiver and its stable uinput device definition.
3. Install any custom source descriptor and set `CONTROLLER_ROUTER_SOURCES_FILE` for the Router process.
4. Choose persistent assignment, backup, and socket paths appropriate to the console.
5. Import known EmulationStation mappings and add your custom source without deleting saved assignments.
6. Start `ControllerRouterDevice` with explicit platform configuration paths.
7. Expose bounded, authenticated setup operations through your project's service.
8. Install the session launch adapter, choose enabled systems, and test input in the actual game.

Provide service startup, shutdown, watchdog, permissions, and logs in your installer. Keep personal configuration separate from replaceable software. Avoid creating a second competing Router process when another compatible integration already owns the merged outputs.

Router recognises RetroPie, Batocera, and Recalbox configuration conventions. That recognition alone does not validate your receiver, emulator wrapper, or installer on every board and version.

Register session routing

Bundle `router_shared.launch` and `router_shared.launch_install` with the engine. Register the launch adapter during installation or upgrade; game hooks should report sessions only. Runtime routing uses temporary settings and leaves saved `retroarch.cfg` files alone.

RetroPie

Call `launch_install.install_retropie()` during installation or upgrade. It backs up replaced Router files, repairs Router-owned profiles, and registers the adapter in existing `emulators.cfg` commands. Repeating installation leaves unchanged files alone.

Batocera

Call `install_batocera()` during installation and `activate_batocera()` at service startup. Installation checks that it recognises the Libretro generator before changing integration files. Startup binds the generator overlay. If the generator layout is unrecognized, installation stops without changing saved RetroArch settings.

Recalbox 10.x

Call `install_recalbox()` during installation and `activate_recalbox()` at service startup. Installation stages a patched Libretro generator in the persistent share; startup mounts it over Recalbox's read-only generator. The generated command runs the adapter through `/usr/bin/python3` because the share is not executable.

Test from EmulationStation, or close its menu before a remote launch. The menu holds the display, so a simultaneous RetroArch launch cannot initialize video.

Another RetroArch launcher

Wrap the original command with the session adapter:

```
python3 -m router_shared.launch --config PATH \
  --retroarch EXECUTABLE -- ORIGINAL_ARGUMENTS
```

If your launcher manages the RetroArch process itself, prepare a temporary settings file instead:

```
python3 -m router_shared.controller_router prepare-launch \
  --platform PLATFORM --config PATH \
  --retroarch EXECUTABLE --output SESSION_FILE
```

Append the generated file last, keep it until RetroArch exits, then delete it. The session adapter handles this file lifecycle, signals, exit status, and output-loss reporting for you.

During play

Keep merged output devices alive throughout the game. Missing or duplicate configured outputs block a routed launch. If Router fails during play, the player must exit and relaunch after recovery. Test both legacy and reservation modes, including keyboard controls, hotkeys, and physical-source reconnects.

Preserve system policy during deployment

New format-2 configurations start with `physical_scope: "nes"`. Upgrades retain `nes`, `all`, or `systems` and any saved `physical_systems` list. Assignment-only requests from older clients preserve the existing policy.

The console adapter and input engine use the same canonical system ID. A disabled system receives no routing override, and physical sources remain ungrabbed for that session. Keep merged output devices alive regardless of system policy. Standalone emulators bypass this integration.

Expose Players and Systems through your authenticated adapter instead of writing configuration files from a browser. See **Per-system routing policy** in the Technical Reference for the fields, and **Setup** in the User Guide for the player workflow.

Protect console configuration

Finish games before installing or reconfiguring virtual controllers. Close EmulationStation on RetroPie before recreating virtual joysticks; its input manager can fail during device replacement.

Router's atomic writes preserve existing permissions and ownership. When writing as root under `/opt/retropie/configs/`, the writer assigns `pi:pi`. Use that helper for managed writes rather than replacing a file through an unrelated root-owned temporary file.

Route mapping changes through the Router integration. Receivers and game-specific wrappers must not independently overwrite the same player indexes. Decide which platform files your adapter owns and document that boundary before deploying it.

RetroArch configuration precedence and upgrades

Router appends its temporary settings after existing appended files. RetroArch can still load a core or game override afterward. A user override containing joypad indexes or reservations can therefore replace the session routing. Preserve unrelated game overrides; inspect controller-specific overrides when a game selects the wrong merged player despite correct launch diagnostics.

During RetroPie installation or upgrade, Router backs up the FCEUmm and Nestopia core overrides and removes only `input_player1_joypad_index` through `input_player4_joypad_index` inside one recognised old Router block. Button bindings, hotkeys, unmarked indexes, and other settings remain. This migration is installer-only. Router never changes these files at receiver startup or game launch, and RetroPie configuration and profile files remain owned by `pi:pi`.

Optional UNO Q runtime

The port-80 chooser links to **Setup** for Players and Systems and to **Pair console** on HTTPS port 8444. Router owns the console registry and authenticated console management connection. Product adapters retain game actions and scoped app credentials. Assignment changes take effect on the next launch.

The `uno_portal` directory supplies the browser chooser, host broker, product lifecycle service, lease coordinator, Matrix scheduler, and App Lab sketch. It is separate from the console wheel.

The current installer expects the `arduino` account with UID 1000, App Lab, Docker Compose, user services, and the existing registered product layout. It is designed for the current VirtualGlove and R.O.B. Vision adapters. A new app must extend that registration; this is not a generic install command for an arbitrary third product.

To add your app:

1. Add a fixed app ID and product path to the portal's registries and validation allowlists.
2. Define its browser port and local health endpoint.
3. Define how its authenticated game-session state becomes a live-game boolean.
4. Register its manifest path and include the display client in its runtime.
5. Add lease enforcement to its input producer and receiver.
6. Add the product to lifecycle startup and installer backup/upgrade handling.
7. Add its chooser entry and direct-visit selection behaviour.
8. Extend tests before enabling it on a UNO Q.

Keep Router as the sole Matrix firmware owner. Product installation and startup should not flash a competing sketch. The shared Matrix's first App Lab build can take several minutes; keep progress visible and explain the wait to users.

Entry-page startup recovery

The broker monitors `http://127.0.0.1/api/state` in a background thread. When the entry API is unavailable, it starts only the `uno-controller-router-entry` Compose project and checks readiness before reporting success. It retries after failure, suppresses repeated identical error messages, and reports Docker port conflicts. Healthy entry containers are left running. Recovery does not restart Docker or product services, change selection, or rewrite console settings. An unrelated listener occupying port 80 requires operator intervention; the broker does not stop it.

Shared device pairing

Services and credentials

Controller Router owns the connection. The host broker serves secure Setup on TCP **8444** and stores schema-1 connections in `/home/arduino/.local/state/controller-router/connections.json`. The console TLS service listens on TCP **55359**.

Router has a management credential. VirtualGlove and R.O.B. Vision have separate credentials that can be revoked independently. Browser responses include only public connection and readiness fields.

Pairing checks

`router_shared.pairing.Peer` verifies the console certificate before sending a code or credential. A CR1 code carries a 100-bit certificate fingerprint prefix and a 60-bit authorization value. The console's pairing window lasts 300 seconds, allows one successful transaction, and locks after five incorrect codes. Matrix confirmation lasts 120 seconds and allows five attempts.

Requests are bounded, servers allow at most 16 concurrent workers, and TLS 1.2 or later is required. Secure browser writes require a matching HTTPS Origin, a fixed action header, and JSON content.

Provisioning and migration

Provisioning follows prepare, commit, and finalize. Private journals keep the previous Router registry and app configuration so an interrupted or failed transaction can restore them. Successful changes retain private before-connection backups.

Legacy migration verifies an HMAC over a fresh nonce, canonical console ID, and observed TLS certificate. Adoption signs the new connection transcript. Hostnames alone do not authorize consolidation; conflicting records require an explicit re-pairing choice.

Product adapters and console API

Products expose `/api/router-pairing` only to local host requests bearing a capability. Keep `data/router-pairing-adapter-token` files out of browser responses. Adapters load app credentials into live caches without replacing ROM registries, calibration, or player settings. Router discovers new console and controller adapters and provisions them through its pinned management connection. Disabled app access remains disabled.

The console helper provides `/identity`, `/pair`, `/legacy-proof`, `/adopt`, `/manage`, and `/router`. `/manage` uses Router's credential for inspection, provisioning, and removal. `/router` uses that credential and `RouterStore` revision checks for assignments and system policy. It does not write saved RetroArch configurations. Buddy's first-pair adapter adds its Player 2 source; emulator configuration migration remains installation-only.

Backups

Back up the controller's connection registry, `tls` directory, and private before-connection backups. On RetroPie, also back up `/var/lib/controller-router/link/{console-id,adapters.json,connection.json,certificate.pem,private-key.pem}` and each product's credential files.

Restore requires the separately installed shared link service. Keep backups private and out of diagnostics and support reports. Never snapshot a provisioning transaction in progress.

Upgrade and rollback design

Store pairing credentials, game registries, and player assignments outside software replacement paths. Back up the current software before changing services. Verify new service and Matrix health; restore previous software if activation fails.

Keep a newer compatible shared Router when a product bundles an older copy, while still registering the new product. A newly installed controller must not disappear merely because shared code did not need upgrading.

Assignment rollback is separate from software rollback. `RouterStore` saves a `.previous` snapshot and requires a current revision for restoration. Your UI should explain which settings are being restored.

Existing integrations as deployment examples

R.O.B. Vision installs the shared console service and a fixed Buddy source descriptor, then keeps Buddy on Player 2 through its adapter. VirtualGlove vendors the same engine behind its signed console API. Both bundle `uno_portal` so their UNO Q installers register their services with the same Matrix owner.

Read the implementation before copying service paths or product assumptions:

- Console: R.O.B. Vision installer and Router setup.
- Controller: VirtualGlove installer and shared UNO installer.

Release checklist

Test a clean install and repeat upgrade. Confirm assignments survive, failed starts roll back, input releases on disconnect, and settings cannot change during a live game.

For the UNO Q runtime, also test a reboot with no selection, automatic game selection, missing or expired leases, display expiry, and a newly registered app alongside a newer Router. Record the platform and device used for each result; distinguish build compatibility, mocked tests, and physical-device tests in release notes.